Министерство науки и высшего образования Российской Федерации

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»

Институт информатики и кибернетики

Кафедра технической кибернетики

Отчет по лабораторной работе №3

Дисциплина: «ООП»

Тема «Дополнение пакета для работы с функциями одной переменной, заданными в табличной форме»

Выполнил: Забанов Захар Денисович

Группа: 6201-120303D

Самара, 2025

Задание 1: Изучение классов исключений

Ход выполнения:

Изучены следующие классы исключений Java API:

- Exception - базовый класс для проверяемых исключений
- IndexOutOfBoundsException - выход за границы коллекции
- ArrayIndexOutOfBoundsException - специализация выхода за границы для массивов
- IllegalArgumentException - некорректный аргумент метода
- IllegalStateException - некорректное состояние объекта

Результат:

Получено понимание иерархии исключений и их назначения для использования в разрабатываемом пакете.

Задание 2: Создание классов исключений

Ход выполнения:

Созданы два специализированных класса исключений в пакете functions:

1. **FunctionPointIndexOutOfBoundsException**
   - Наследуется от IndexOutOfBoundsException
   - Предназначено для случаев выхода за границы набора точек
2. **InappropriateFunctionPointException**
   - Наследуется от Exception
   - Используется при нарушении упорядоченности точек

Результат:

```
// FunctionPointIndexOutOfBoundsException.java

public class FunctionPointIndexOutOfBoundsException

   extends IndexOutOfBoundsException { ... }



// InappropriateFunctionPointException.java

public class InappropriateFunctionPointException

   extends Exception { ... }
```

Задание 3: Модификация TabulatedFunction

Ход выполнения:

Класс TabulatedFunction из лабораторной работы №2 модифицирован:

1. **Конструкторы** теперь выбрасывают IllegalArgumentException при:
   - Левая граница ≥ правой границы
   - Количество точек < 2
2. **Методы доступа** к точкам выбрасывают FunctionPointIndexOutOfBoundsException при неверном индексе
3. **Методы изменения** выбрасывают InappropriateFunctionPointException при нарушении упорядоченности
4. **Метод deletePoint()** выбрасывает IllegalStateException при попытке удаления, если останется менее 2 точек

Результат:

Все методы класса теперь корректно обрабатывают некорректные ситуации через механизм исключений.

Задание 4: Реализация LinkedListTabulatedFunction

Ход выполнения:

Реализован новый класс, использующий двусвязный циклический список:

1. **Внутренний класс FunctionNode**:
   - Содержит FunctionPoint, ссылки на предыдущий и следующий узлы
   - Приватный статический класс для обеспечения инкапсуляции
2. **Структура списка**:
   - Циклический двусвязный список с выделенной головой (head)
   - Голова не содержит данных, всегда присутствует в списке
3. **Оптимизированные методы**:
   - getNodeByIndex() - кэширование последнего доступа
   - addNodeToTail(), addNodeByIndex(), deleteNodeByIndex()

Результат:

Полностью функционирующая реализация табулированной функции на основе связного списка.

Задание 5: Реализация методов табулированной функции

Ход выполнения:

Для LinkedListTabulatedFunction реализованы те же методы, что и для массива:

- Конструкторы с аналогичными параметрами
- Методы интерфейса TabulatedFunction
- Такие же исключения в аналогичных ситуациях

Результат:

Обе реализации (ArrayTabulatedFunction и LinkedListTabulatedFunction) предоставляют идентичный API.

Задание 6: Создание интерфейса и рефакторинг

Ход выполнения:

1. Создан интерфейс TabulatedFunction с объявлениями всех общих методов
2. Класс TabulatedFunction переименован в ArrayTabulatedFunction
3. Оба класса реализуют созданный интерфейс

Результат:

```
public interface TabulatedFunction {

   int getPointsCount();

   double getLeftDomainBorder();

   // ... другие методы

}
```

```
public class ArrayTabulatedFunction implements TabulatedFunction { ... }

public class LinkedListTabulatedFunction implements TabulatedFunction { ... }
```

Задание 7: Тестирование

Ход выполнения:

Создан расширенный класс Main для тестирования:

1. Проверка всех исключений для обеих реализаций
2. Тестирование через интерфейс TabulatedFunction
3. Сравнение результатов обеих реализаций

Результат выполнения программы:

=== Тестирование ArrayTabulatedFunction ===

1. Тест некорректных конструкторов:

Перехвачено ожидаемое исключение: Левая граница должна быть меньше правой

Перехвачено ожидаемое исключение: Количество точек должно быть не менее 2

2. Создание корректной функции f(x) = x^2:

Точки функции:

[0] (0.0000; 0.0000)

[1] (1.0000; 1.0000)

[2] (2.0000; 4.0000)

[3] (3.0000; 9.0000)

[4] (4.0000; 16.0000)

3. Тест выхода за границы индекса:

Перехвачено ожидаемое исключение: Индекс 10 вне границ [0, 4]

4. Тест нарушения упорядоченности:

Перехвачено ожидаемое исключение: X-координата (0.5) должна быть больше предыдущей (1.0)

5. Тест удаления точек:

После удаления точки 1:

Точки функции:

[0] (0.0000; 0.0000)

[1] (2.0000; 4.0000)

[2] (3.0000; 9.0000)

[3] (4.0000; 16.0000)

Перехвачено ожидаемое исключение: Нельзя удалить точку: функция должна содержать минимум 2 точки

6. Тест добавления дублирующей точки:

Перехвачено ожидаемое исключение: Точка с x=2.0 уже существует

=== Тестирование LinkedListTabulatedFunction ===

1. Создание функции f(x) = sin(x):

Точки функции:

[0] (0.0000; 0.0000)

[1] (0.7854; 0.7071)

[2] (1.5708; 1.0000)

[3] (2.3562; 0.7071)

[4] (3.1416; 0.0000)

2. Тест добавления точки:

После добавления точки (1.5, sin(1.5)):

Точки функции:

[0] (0.0000; 0.0000)

[1] (0.7854; 0.7071)

[2] (1.5000; 0.9975)

[3] (1.5708; 1.0000)

[4] (2.3562; 0.7071)

[5] (3.1416; 0.0000)

3. Тест удаления точки:

После удаления точки 2:

Точки функции:

[0] (0.0000; 0.0000)

[1] (0.7854; 0.7071)

[2] (1.5708; 1.0000)

[3] (2.3562; 0.7071)

[4] (3.1416; 0.0000)

4. Тест вычисления значения:

f(0.50) = 0.4794

f(1.00) = 0.8415

f(2.00) = 0.9093

f(2.50) = 0.5985

=== Тестирование через интерфейс ===

ArrayTabulatedFunction: 3 точек

LinkedListTabulatedFunction: 3 точек

Сравнение значений в точке x=1.0:

ArrayTabulatedFunction: f(1.0) = 2.0

LinkedListTabulatedFunction: f(1.0) = 2.0